

LAPORAN ROBOCODE TANK ROYALE MENGUNAKAN ALGORITMA GREEDY

untuk:

Tugas Besar Mata Kuliah Strategi Algoritma



Dipersiapkan Oleh:

Nasywa Putri Salsabila	124140005
Angelica Putri	124140131
Amelinda Simanjuntak	124140038

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
TAHUN 2026**

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan tugas besar mata kuliah Strategi Algoritma yang berjudul “Robocode Tank Royale Menggunakan Algoritma Greedy” ini dapat diselesaikan tepat pada waktunya.

Laporan ini dibuat sebagai salah satu bentuk pemenuhan tugas besar mata kuliah Strategi Algoritma pada Program Studi Teknik Informatika, Fakultas Teknologi Industri, Institut Teknologi Sumatera. Tujuan penulisan laporan ini adalah untuk mendokumentasikan proses implementasi algoritma greedy dalam pembuatan bot Robocode Tank Royale beserta hasil pengujian dan analisisnya.

Kami sadar bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun dari berbagai pihak sangat kami harapkan untuk perbaikan kedepannya. Kami juga mengucapkan terima kasih kepada semua pihak yang telah membantu kelancaran pengerjaan tugas ini, terutama kepada dosen pengampu mata kuliah Strategi Algoritma serta para asisten laboratorium yang telah memberikan bimbingan dan arahan.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca, khususnya bagi mahasiswa yang tertarik untuk mengembangkan strategi algoritma pada permainan Robocode Tank Royale.

Lampung Selatan, 22 Mei 2026

DAFTAR ISI

KATA PENGANTAR.....	1
BAB I	
DESKRIPSI TUGAS.....	4
1.1 Latar Belakang.....	4
1.2 Deskripsi Tugas.....	5
1.3 Tujuan.....	6
BAB II	
LANDASAN TEORI.....	7
2.1 Algoritma Greedy Secara Umum.....	7
2.1.1 Pengertian Algoritma Greedy.....	7
2.1.2 Karakteristik algoritma greedy.....	7
2.1.3 Komponen algoritma greedy.....	8
2.2 Cara Kerja Program Secara Umum.....	8
2.2.1 Mekanisme Dasar Bot.....	8
2.2.2 Implementasi Algoritma Greedy dalam Bot.....	9
2.2.3 Cara Menjalankan Bot.....	10
BAB III	
APLIKASI STRATEGI GREEDY.....	11
3.1 Mapping Persoalan Robocode ke Elemen Greedy.....	11
3.2 Empat Alternatif Solusi Greedy.....	12
3.2.1 Alternatif Bot 1 - Alt 1: Survival Bot (Greedy Statis Berbasis Pojok Lapangan)....	12
3.2.2 Alternatif Bot 2 – Alt 2: Adaptive Bot (Greedy Dinamis Berbasis Level Energi Sendiri).....	12
3.2.3 Alternatif Bot 3 – Alt 3: Flanker Bot (Greedy Serangan Samping & Prediksi Lintasan).....	13
3.2.4 Main Bot - Opportunist Bot (Greedy Berbasis Jumlah Musuh Hidup & Manajemen Jarak).....	14
3.3 Analisis Efisiensi dan Efektivitas.....	14
3.4 Strategi Greedy yang Dipilih.....	15
BAB IV	
IMPLEMENTASI DAN PENGUJIAN.....	17
4.1 Implementasi Empat Alternatif Solusi.....	17
4.1.1 Pseudocode Bot Utama.....	17
4.1.2 Pseudocode Bot Alternatif 1.....	21
4.1.3 Pseudocode Bot Alternatif 2.....	24
4.1.4 Pseudocode Bot Alternatif 3.....	26
4.2 Struktur Data, Fungsi, dan Prosedur.....	30
4.2.1 Struktur Data.....	30

4.2.2 Fungsi.....	31
4.2.3 Prosedur.....	32
4.3 Pengujian Program.....	34
4.3.1 Pengujian 1.....	35
4.3.2 Pengujian 2.....	36
4.3.3 Pengujian 3.....	37
4.3.4 Pengujian 4.....	38
4.4 Analisis dan Hasil Pengujian.....	38
BAB V	
KESIMPULAN DAN SARAN.....	41
5.1 Kesimpulan.....	41
5.2 Saran.....	42
LAMPIRAN.....	44
DAFTAR PUSTAKA.....	45

DAFTAR GAMBAR

Gambar 4.1 Hasil Pengujian 1.....	35
Gambar 4.2 Hasil Pengujian 2.....	36
Gambar 4.3 Hasil Pengujian.....	37
Gambar 4.4 Hasil Pengujian 4.....	38

DAFTAR TABEL

Tabel 4.1 Analisis Pengujian 1.....	35
Tabel 4.2 Analisis Pengujian 2.....	36
Tabel 4.3 Analisis Pengujian 3.....	37
Tabel 4.4 Analisis Pengujian 4.....	38

BAB I

DESKRIPSI TUGAS

1.1 Latar Belakang

Robocode Tank Royale merupakan platform simulasi pertempuran antar tank yang dibangun untuk menguji kemampuan pemrograman dan strategi algoritma. Pada platform, setiap programmer tidak mengendalikan tank secara langsung, tetapi menuliskan dulu kode program yang menjadi “otak” dari tank. Tank yang telah diprogram akan bertarung secara otonom di dalam sebuah arena sampai menyisakan satu pemenang, sama seperti konsep battle royale umumnya. Robocode sendiri merupakan singkatan dari “Robot code”, yang merujuk pada esensi permainan ini sebagai ajang pembuatan kode untuk robot (Robocode SourceForge, 2024). Robocode Tank Royale merupakan versi terbaru yang mendorong bot-bot untuk saling bertarung melalui jaringan internet (Robocode Dev, 2024).

Salah satu tantangan besar dalam pengembangan Robocode adalah bagaimana membuat bot yang mampu mengambil keputusan secara cepat dan tepat pada setiap putaran. Setiap putaran dalam Robocode adalah unit waktu terkecil di mana bot harus menentukan aksi seperti bergerak, memutar radar, membidik, dan menembak. Jika bot gagal mengirimkan perintah tepat waktu, karena proses komputasinya lama, maka bot akan kehilangan kesempatan untuk bertindak pada putaran tersebut. Kondisi ini yang melibatkan algoritma yang tidak hanya efektif secara strategi, tetapi juga efisien dari segi waktu komputasi.

Algoritma greedy bekerja dengan prinsip mengambil keputusan terbaik yang tersedia pada saat itu juga, tanpa memikirkan konsekuensi jangka panjang. Keputusan tersebut disebut dengan optimum lokal. Walaupun sederhana, dengan ini sering kali menghasilkan solusi yang cukup baik, terutama untuk persoalan yang membutuhkan respons cepat (Munir, 2016). Dalam Robocode, algoritma greedy dapat diimplementasikan untuk berbagai aspek, seperti pemilihan target, penentuan kekuatan tembakan, dan penentuan arah gerak tank. Pendekatan greedy dinilai "cukup menjanjikan untuk diterapkan dalam permainan karena kesederhanaan dan kecepatannya" (Jurnal Ilmubersama, n.d.). Selain itu, keberhasilan algoritma greedy terletak pada perumusan fungsi seleksi atau heuristic yang tepat. Heuristic yang digunakan harus mampu merepresentasikan kondisi pertempuran secara akurat agar keputusan yang diambil benar-benar

menguntungkan (E-Jurnal UNAMA, n.d.). Meskipun algoritma greedy tergolong sederhana, efektivitasnya sangat bergantung terhadap pemahaman terhadap problem domain.

Tugas besar ini merupakan implementasi algoritma greedy dalam pembuatan bot Robocode Tank Royale. Membuat empat bot dengan heuristic greedy yang berbeda-beda. Keempat bot kemudian akan diuji coba dan dianalisis untuk menentukan strategi greedy mana yang paling efektif dalam memenangkan pertandingan. Dengan demikian, laporan ini diharapkan dapat memberikan gambaran yang lebih komprehensif mengenai penerapan algoritma greedy dalam lingkungan yang dinamis dan kompetitif.

1.2 Deskripsi Tugas

Adapun rincian tugas yang dilaksanakan adalah sebagai berikut:

1. Membuat 4 bot (1 utama dan 3 alternatif) dalam bahasa C# (.NET) yang mengimplementasikan algoritma greedy pada bot permainan Robocode Tank Royale dengan tujuan memenangkan permainan. Tetap dikumpulkan 1 bot yang terbaik saja.
2. Strategi greedy yang diimplementasikan setiap kelompok dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dilakukan dengan mengoptimalkan komponen skor (*Bullet Damage*, *Bullet Damage Bonus*, *Survival Score*, *Last Survival Bonus*, *Ram Damage*, *Ram Damage Bonus*).
3. Strategi greedy yang diimplementasikan berbeda untuk setiap bot dan setiap strategi greedy menggunakan heuristic yang berbeda.
4. Bot yang dibuat TIDAK BOLEH sama dengan sampel yang diberikan sebagai contoh, baik dari starter pack maupun dari repository engine asli.
5. Melakukan pengujian minimal 3 kali pertandingan antara bot utama dengan bot asisten (format 1 vs 1).
6. Menganalisis efektivitas masing-masing strategi greedy.
7. Mendokumentasikan seluruh proses implementasi dan hasil pengujian dalam laporan.

1.3 Tujuan

Tujuan dari pengerjaan tugas meliputi:

1. Memahami konsep algoritma greedy secara mendalam, termasuk karakteristik dan komponen-komponennya.
2. Mengimplementasikan strategi greedy dalam konteks pengembangan bot pertarungan Robocode Tank Royale.
3. Menganalisis efektivitas strategi greedy melalui pengujian dan perbandingan antar variasi strategi.
4. Mengevaluasi kelebihan dan kelemahan pendekatan greedy dalam pengambilan keputusan real-time.
5. Menghasilkan bot terbaik yang mampu berkompetisi dengan bot dari kelompok lain.

BAB II

LANDASAN TEORI

2.1 Algoritma Greedy Secara Umum

2.1.1 Pengertian Algoritma Greedy

Algoritma greedy merupakan salah satu pendekatan klasik dalam pemecahan masalah yang termasuk ke dalam ranah algorithm design paradigm. Istilah "greedy" secara harfiah berarti rakus atau tamak, yang secara metaforis menggambarkan perilaku algoritma ini yang selalu ingin mengambil yang terbaik pada setiap kesempatan. Algoritma greedy membangun solusi langkah demi langkah. Pada setiap langkah, algoritma ini akan memilih opsi yang memberikan keuntungan paling besar pada saat itu juga, tanpa mempertimbangkan apakah pilihan tersebut akan menguntungkan atau merugikan di masa depan.

Pendekatan ini sangat berbeda dengan algoritma divide and conquer atau dynamic programming yang cenderung memandang masalah secara menyeluruh. Greedy lebih mengutamakan kecepatan dan kesederhanaan. Karena tidak melakukan backtracking atau penelusuran mundur, algoritma greedy dapat menghasilkan solusi dalam waktu yang relatif singkat. Namun, konsekuensinya adalah solusi yang dihasilkan belum tentu merupakan solusi global yang paling optimal. Dalam banyak kasus, greedy hanya mampu menemukan solusi yang "cukup baik" atau mendekati optimal (Munir, 2016).

Prinsip utama dari algoritma greedy adalah *"take what you can get now"*, yaitu mengambil pilihan yang terbaik yang dapat diperoleh pada saat itu tanpa memperhatikan konsekuensi ke depan (Sianaeng, Kewo, & Suwanto, 2015).

2.1.2 Karakteristik algoritma greedy

Sejumlah karakteristik khas dapat diidentifikasi dari algoritma greedy. Pertama, algoritma greedy selalu membuat pilihan optimal lokal pada setiap langkah, yaitu pilihan terbaik berdasarkan informasi yang tersedia pada saat itu tanpa melihat ke depan. Kedua, algoritma greedy tidak melakukan backtracking setelah sebuah keputusan diambil, keputusan tersebut tidak akan pernah diubah atau dibatalkan meskipun di kemudian hari ternyata keputusan tersebut kurang tepat. Ketiga, algoritma greedy berorientasi pada kecepatan karena tidak adanya proses penelusuran ulang sehingga waktu komputasi yang dibutuhkan relatif kecil (Munir, 2018).

Karakteristik-karakteristik tersebut menjadikan algoritma greedy sebagai pilihan yang sangat populer untuk persoalan optimasi yang memiliki struktur matroid atau greedy-choice property (Jurnal UMNU, n.d.). Keberhasilan algoritma greedy sangat bergantung pada perumusan fungsi seleksi. Fungsi seleksi yang kurang tepat akan mengarahkan algoritma ke solusi yang buruk, meskipun algoritma dijalankan dengan benar (Jurnal PNJ, n.d.).

2.1.3 Komponen algoritma greedy

Algoritma greedy memiliki komponen-komponen utama yang menjadi kerangka kerja untuk merancang solusi greedy dalam berbagai domain persoalan. Kelima komponen tersebut adalah himpunan kandidat, himpunan solusi, fungsi seleksi, fungsi kelayakan, dan fungsi objektif. Himpunan kandidat adalah kumpulan semua kemungkinan pilihan yang tersedia pada suatu tahap penyelesaian. Himpunan solusi adalah kumpulan pilihan yang telah diambil dan membentuk solusi lengkap. Fungsi seleksi adalah kriteria yang digunakan untuk memilih kandidat terbaik dari himpunan kandidat. Fungsi kelayakan adalah pemeriksaan apakah suatu kandidat yang dipilih layak untuk dimasukkan ke dalam himpunan solusi. Fungsi objektif adalah fungsi yang menentukan nilai atau kualitas dari suatu solusi, di mana dalam persoalan maksimisasi fungsi objektif akan memberikan nilai yang lebih tinggi untuk solusi yang lebih baik (E-Jurnal Mikroskil, n.d.).

2.2 Cara Kerja Program Secara Umum

2.2.1 Mekanisme Dasar Bot

Setiap bot dalam Robocode Tank Royale beroperasi dalam sebuah loop tak terbatas yang disebut dengan game loop. Setiap iterasi dalam loop tersebut mewakili satu turn. Pada setiap turn, bot dapat melakukan tiga jenis aksi utama, yaitu menggerakkan tubuh (*body*), memutar meriam (*gun*), serta memutar radar. Selain itu, bot juga dapat menembakkan peluru jika kondisi meriam sudah dingin. Informasi mengenai posisi, energi, dan arah hadap bot akan selalu diperbarui secara otomatis oleh game engine di setiap awal turn. Setiap bot memiliki batas waktu yang sangat ketat untuk menyelesaikan proses berpikirnya. Batas waktu ini disebut turn timeout, yang secara default diatur antara 30 hingga 50 milidetik. Jika bot tidak mengirimkan perintah aksinya sebelum batas waktu habis, maka bot akan kehilangan kesempatan untuk bergerak atau menembak pada turn tersebut. Ketentuan ini menjadi alasan utama mengapa algoritma greedy

dipilih, karena greedy mampu memberikan keputusan dalam waktu yang sangat singkat (Robocode Dev, 2024)

Komunikasi antara bot dan game engine dilakukan melalui protokol WebSocket. Bot akan mengirimkan intent (rencana aksi) berupa perintah-perintah seperti *SetForward*, *SetTurnRight*, *Fire*, dan sebagainya. Di balik layar, API bot secara otomatis mengirimkan intent tersebut ke server sehingga pengembang bot tidak perlu memusingkan detail teknis komunikasi jaringan (Robocode Tank Royale GitHub, 2024).

Selain perintah-perintah dasar, bot juga harus merespons berbagai kejadian atau event yang terjadi selama pertandingan. Misalnya, *event OnScannedBot* akan dipicu ketika radar bot berhasil mendeteksi musuh. Di dalam *event handler* inilah logika pemilihan target biasanya ditempatkan. Demikian pula dengan *event OnHitByBullet* yang dipicu ketika bot terkena tembakan musuh. Event tersebut dapat dimanfaatkan untuk melakukan gerakan menghindar atau evasif (RoboWiki, 2024).

2.2.2 Implementasi Algoritma Greedy dalam Bot

Implementasi algoritma greedy dalam bot Robocode dasarnya adalah tentang merumuskan fungsi seleksi yang tepat untuk setiap pengambilan keputusan. Contohnya, saat bot harus memilih target tembak, ia akan menghitung sebuah skor untuk setiap musuh yang terdeteksi. Musuh dengan skor tertinggi akan dipilih sebagai target. Proses perhitungan skor yang akan merepresentasikan prinsip greedy. Fungsi seleksi dapat dirumuskan dengan berbagai parameter, seperti jarak antara bot dengan musuh, energi musuh saat ini, kecepatan gerak musuh, dan kombinasi dari beberapa parameter sekaligus.

Pemilihan parameter yang relevan yang menentukan apakah bot akan bertindak cerdas atau justru sebaliknya di arena pertempuran. Selain untuk pemilihan target, algoritma greedy dapat digunakan untuk menentukan kekuatan tembakan. Bot dapat memilih power tembakan yang paling menguntungkan berdasarkan jarak ke target dan sisa energi sendiri. Demikian pula untuk keputusan bergerak, bot dapat memilih arah yang menjauh dari konsentrasi musuh terdekat. Pada intinya, setiap keputusan yang diambil bot selalu didasarkan pada pilihan terbaik menurut suatu kriteria tertentu pada saat itu juga (E-Jurnal UNAMA, n.d.)

2.2.3 Cara Menjalankan Bot

Sebelum bot dapat dijalankan, terdapat beberapa tahapan persiapan yang harus dilakukan. Pertama, bot harus memiliki file konfigurasi dalam format JSON. File ini berisi metadata bot seperti nama, versi, daftar penulis, dan deskripsi singkat tentang strategi yang digunakan. Kedua, bot harus memiliki file kode sumber C# yang mewarisi kelas Bot dari namespace Robocode.TankRoyale.BotApi. Ketiga, bot harus memiliki file proyek .csproj yang mereferensikan paket Robocode.TankRoyale.BotApi versi 0.30.0. Setelah semua file disiapkan, langkah selanjutnya adalah menjalankan *game engine* dengan perintah `java -jar robocode-tankroyale-gui-0.30.0.jar`. Melalui antarmuka grafis *game engine*, pengguna dapat melakukan konfigurasi direktori tempat bot disimpan, melakukan boot terhadap bot yang ingin diikuti, dan terakhir memulai pertempuran dengan mengklik tombol "*Start Battle*" (Robocode Dev, 2024).

BAB III

APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode ke Elemen Greedy

Dalam Robocode, strategi Greedy biasanya diterapkan pada salah satu dari dua hal yaitu membidik musuh atau pergerakan untuk menghindari atau mengambil energi. Komponen pemetaan persoalan Robocode kedalam 5 algoritma Greedy berikut:

1. **Himpunan kandidat** : Himpunan yang berisi seluruh robot musuh yang saat ini terdeteksi oleh radar robot kita melalui fungsi `onScannedRobot`. Jika terdapat lebih dari satu musuh yang tertangkap radar, maka seluruh musuh tersebut masuk ke dalam daftar kandidat yang bisa dipilih.
2. **Himpunan solusi** : Himpunan yang berisi satu robot musuh spesifik yang akhirnya terpilih untuk dikunci oleh radar (*radar lock*) dan dijadikan target tembakan utama pada *tick* atau putaran (*turn*).
3. **Fungsi seleksi** : Mekanisme atau kriteria yang digunakan untuk memilih satu target terbaik dari himpunan kandidat. Di sini digunakan prinsip Greedy on Distance (Jarak Terdekat), yaitu memilih musuh yang memiliki jarak paling dekat dengan robot kita (`e.getDistance()`). Logika *greedy*-nya adalah dengan memilih musuh terdekat, maka waktu tempuh peluru menjadi lebih singkat dan akurasi tembakan menjadi jauh lebih tinggi.
4. **Fungsi kelayakan** : Fungsi untuk memeriksa apakah target yang sudah dipilih oleh fungsi seleksi benar-benar layak dan aman untuk dieksekusi tindakan penembakan. Pengecekan ini meliputi batasan (*constraints*) internal robot, seperti: Apakah senjata tidak sedang panas (`getGunHeat() == 0`). Apakah energi robot kita masih cukup untuk menembak ($\text{energi} > 0,1$), karena menembak di Robocode akan mengurangi energi sendiri. Jika kondisi tersebut terpenuhi, maka target dinyatakan layak.
5. **Fungsi Objektif** : Tujuan akhir yang ingin dicapai dari seluruh rangkaian proses *greedy* ini, yaitu memaksimalkan kerusakan (*damage*) yang dihasilkan dari tembakan peluru ke musuh dan mengeliminasi target secepat mungkin untuk memenangkan pertandingan dengan skor tertinggi.

3.2 Empat Alternatif Solusi Greedy

3.2.1 Alternatif Bot 1 - Alt 1: Survival Bot (Greedy Statis Berbasis Pojok Lapangan)

Bot ini mengimplementasikan strategi *greedy* posisi instan untuk mencari zona paling aman di awal laga demi memprioritaskan ketahanan hidup tertinggi.

- **Inisialisasi Posisi:** Begitu ronde dimulai, fungsi MoveToCorner() langsung memaksa bot bergerak lurus menuju koordinat sudut spesifik, yaitu target (60, 60).
- **Loop Utama:** Setelah menetap di sudut, jika radar mendeteksi musuh (enemyDetected == true), ia memicu AttackMode(). Jika kehilangan jejak, ia masuk ke SurvivalMode() yang hanya melakukan gerakan maju-mundur kecil di sekitar area sudut tersebut (Forward(80), TurnRight(30)).
- **Kalkulasi Peluru:** Dilengkapi fungsi CalculateFirePower() yang *greedy* terhadap kondisi musuh; jika energi musuh kritis (< 20) atau jaraknya sangat dekat (< 100), bot langsung melepaskan tembakan daya penuh Fire(3).
- **Kelebihan:** Sangat konsisten dalam memenangkan status juara bertahan hidup (*Survival Score*) pada ronde-ronde awal karena posisinya di sudut meminimalkan sudut arah datangnya serangan musuh.
- **Kekurangan:** Pola pergerakannya sangat kaku, pasif, dan mudah terjebak atau terpojok apabila musuh yang bertipe agresif berhasil mendeteksi dan menyerang koordinat sudut tersebut.

3.2.2 Alternatif Bot 2 – Alt 2: Adaptive Bot (Greedy Dinamis Berbasis Level Energi Sendiri)

Bot ini mengambil keputusan taktis secara *greedy* dengan mengevaluasi kondisi kesehatannya sendiri (*Self-Energy*) pada setiap putaran secara dinamis.

- **Fase Energi Tinggi (Energy > 70):** Bot berfokus mengamankan posisi di tepi lapangan perbatasan melalui MoveToEdge(). Jika mendeteksi musuh di jarak dekat (distance < 150), ia langsung mengeksekusi tembakan mematikan Fire(3) dan bergerak maju menabrak (Forward(50)).
- **Fase Energi Sedang (Energy > 30):** Bot beralih ke mode menghindar menggunakan gerakan ZigZag() dan menurunkan daya tembak maksimalnya menjadi Fire(2) hanya jika jarak musuh berada di bawah 200 unit.

- **Fase Energi Kritis (Energy ≤ 30):** Bot mengaktifkan ChaosMove(), bergerak acak menggunakan fungsi random (rnd.Next) ke arah depan atau belakang demi mengulur waktu bertahan hidup sembari melarikan diri menjauh.
- **Kelebihan:** Pengambilan keputusan pergerakan dan daya tembak menjadi jauh lebih dinamis serta adaptif karena disesuaikan dengan kekuatan fisik riil bot saat itu.
- **Kekurangan:** Kalkulasi strateginya murni bersifat jangka pendek (lokal). Ketika energinya berada di fase kritis, gerakan acak dari ChaosMove() berpotensi membuat bot menabrak dinding atau justru tidak sengaja mendekati musuh.

3.2.3 Alternatif Bot 3 – Alt 3: Flanker Bot (Greedy Serangan Samping & Prediksi Lintasan)

Bot ketiga menerapkan strategi *greedy* yang berorientasi penuh pada raihan poin kerusakan serangan (ofensif) melalui taktik kepungan samping.

- **Metode Gerakan (Flanking):** Bot memanfaatkan variabel arah flankSide (bernilai 1 atau -1). Jika jarak musuh jauh (> 250), ia mendekat dari samping (ApproachFromSide). Jika jarak sudah ideal, ia akan mengitari musuh (OrbitEnemy) dengan sudut tegak lurus bearing + (flankSide * 90) untuk menghindari peluru lawan.
- **Tembakan Prediktif (Predictive Shoot):** Berbeda dengan bot lainnya, bot ini menghitung fisika peluru secara *greedy*. Ia mengukur waktu tempuh peluru (distance / bulletSpeed) lalu memprediksi koordinat masa depan musuh (futureX, futureY) berdasarkan kecepatan (evt.Speed) dan arah pergerakan musuh (evt.Direction) agar peluru yang ditembakkan tidak meleset.
- **Kelebihan:** Memiliki akurasi serangan yang sangat tinggi serta mampu menghasilkan poin kerusakan (*Bullet Damage* dan *Bullet Bonus*) yang masif karena menembak ke posisi masa depan musuh.
- **Kekurangan:** Sangat boros energi karena terus-menerus memicu tembakan prediktif berdaya tinggi (PredictiveShoot(3)) secara agresif, sehingga rentan mati lebih cepat jika musuh tiba-tiba mengubah arah gerakannya secara acak.

3.2.4 Main Bot - Opportunist Bot (Greedy Berbasis Jumlah Musuh Hidup & Manajemen Jarak)

Bot utama ini menggunakan strategi *greedy* berdasarkan informasi global jumlah musuh yang masih hidup di arena pertempuran ($\text{aliveEnemies} = \text{totalEnemies} - \text{deadEnemies}$).

- **Fase Awal/Ramai ($\text{aliveEnemies} > 5$):** Bot mengaktifkan `WaitingMode()`. Pergerakannya cenderung defensif (menjauh jika jarak musuh < 180 unit) dan *greedy* dalam menghemat energi peluru, karena hanya akan menembak jika musuh mendekat atau saat energi musuh sudah sekarat di bawah 35 ($\text{enemyEnergy} < 35$).
- **Fase Transisi (aliveEnemies antara 2 sampai 5):** Bot beralih ke `PositioningMode()` guna menjaga jarak aman ideal (antara 120 hingga 300 unit) dari musuh.
- **Fase Duel Akhir ($\text{aliveEnemies} == 1$):** Bot berubah drastis menjadi sangat agresif di `HunterMode()`, di mana ia langsung berputar ke arah musuh (`TurnLeft(bearing)`) dan maju menerobos sejauh 100 unit untuk menghabisi target terakhir demi mengamankan bonus kemenangan.
- **Kelebihan:** Sangat efisien dalam menghemat energi di awal permainan, meminimalkan risiko terjebak dalam baku tembak massal, dan mampu mengamankan poin *survival* dengan optimal.
- **Kekurangan:** Karena cenderung pasif dan menjauh di awal ronde, bot ini berpotensi kehilangan banyak peluang untuk mengumpulkan poin akumulasi kerusakan senjata (*Bullet Damage*) sejak dini.

3.3 Analisis Efisiensi dan Efektivitas

Berdasarkan hasil analisis dari keempat bot, setiap bot memiliki kekurangan dan kelebihan masing-masing. Jika ditinjau lebih lanjut berdasarkan implementasi kode program, tingkat efisiensi dan efektivitas dari masing-masing strategi bot dapat dijabarkan sesuai urutan pengujian berikut:

- **Alternatif Bot 1 (Alt 1 - Survival Bot)** menunjukkan tingkat efisiensi pertahanan yang tinggi namun memiliki efektivitas pertempuran yang sangat rendah. Dengan langsung mengunci posisi statis di koordinat sudut (*cornering*) (60, 60) sejak awal laga, bot ini meminimalkan paparan sudut serangan musuh secara instan, tetapi pergerakan maju-mundur kecil di area tersebut membuatnya menjadi target pasif yang sangat kaku dan mudah terpojok jika lokasinya berhasil terbaca oleh lawan yang agresif.

- **Alternatif Bot 2 (Alt 2 - Adaptive Bot)** menawarkan efisiensi yang bersifat kondisional dan dinamis karena keputusannya murni bergantung pada sisa kesehatannya sendiri (*self-energy*). Bot ini menjadi sangat boros dan tidak efisien saat energinya melimpah karena terus memicu tembakan berdaya maksimal, namun efektivitas gerakannya dalam mengecoh musuh cukup baik berkat perubahan taktik dari tepi lapangan, manuver zig-zag, hingga gerakan acak (*chaos move*) saat kondisinya kritis.
- **Alternatif Bot 3 (Alt 3 - Flanker Bot)** mengorbankan efisiensi energinya demi mencapai efektivitas serangan murni yang sangat tinggi. Melalui taktik kepungan (*flanking*) dan pengejaran konstan, bot ini secara agresif menguras energinya untuk melepaskan tembakan prediktif. Meskipun memiliki risiko eliminasi awal yang tinggi akibat boros energi, kalkulasi matematis bot ini dalam memprediksi koordinat masa depan musuh berdasarkan arah dan kecepatan gerak target terbukti paling efektif dalam mengumpulkan akumulasi poin kerusakan senjata (*Bullet Damage*).
- **Main Bot (Opportunist Bot)** menjadi strategi yang paling seimbang dalam hal efisiensi dan efektivitas jangka panjang karena memanfaatkan informasi global pertempuran secara *greedy*. Pada fase awal yang ramai, bot ini sangat efisien dalam menjaga cadangan energi karena menahan diri untuk tidak menyerang secara agresif dan memilih menjauh jika musuh mendekat di bawah jarak 180 unit. Perilaku ini sangat efektif untuk mengamankan poin *survival*, yang kemudian ditransisikan secara taktis menjadi mode berburu (*Hunter Mode*) yang agresif saat situasi menyisakan duel satu lawan satu demi mengunci bonus kemenangan.

3.4 Strategi Greedy yang Dipilih

Berdasarkan serangkaian hasil pengujian, analisis efisiensi, serta simulasi tanding komparatif yang telah dilakukan terhadap keempat kandidat bot, Kelompok TinkTank secara bulat memilih **Main Bot (Opportunist Bot)** sebagai strategi *Greedy* utama yang diunggulkan untuk turnamen. Adapun beberapa landasan teoretis dan eksperimental yang mendasari keputusan pemilihan ini adalah sebagai berikut:

1. Penerapan Fungsi Seleksi Heuristic Secara Global (Dinamis)

Berbeda dengan alternatif bot lainnya yang hanya mengandalkan informasi lokal statis (seperti koordinat sudut pada *Survival Bot* atau kondisi internal pada *Adaptive Bot*), *Opportunist Bot* bekerja secara cerdas dengan memanfaatkan informasi makro/global di arena pertempuran, yaitu melalui parameter variabel `aliveEnemies`. Hal ini membuat fungsi seleksi *greedy* objek permainan menjadi jauh lebih dinamis dan fleksibel menyesuaikan risiko turnamen dari waktu ke waktu.

2. Keseimbangan (Trade-off) antara Efisiensi Energi dan Efektivitas Skor

Dalam turnamen Robocode, memenangkan pertempuran tidak hanya ditentukan dari seberapa besar kerusakan (*damage*) yang dihasilkan, melainkan seberapa lama bot mampu bertahan hidup (*survival rate*) untuk mengamankan bonus poin kelipatan.

- Pada awal laga saat musuh masih sangat ramai (> 5 bot), *Opportunist Bot* secara *greedy* menahan diri dari baku hantam massal dan menghemat energi via `WaitingMode()`. Strategi ini mengeliminasi kelemahan utama *Flanker Bot* yang sangat boros energi di awal laga.
- Namun, bot ini tidak sekadar pasif seperti *Survival Bot* (yang terus bersembunyi di sudut). Begitu jumlah musuh menyusut, tingkat keagresifan bot langsung meningkat secara bertahap demi mencuri poin akumulasi kerusakan pada musuh yang sekarat.

3. Optimalisasi Bonus Kemenangan (Survival Score) pada Fase Duel

Keunggulan mutlak dari strategi *Opportunist Bot* terletak pada transisi otomatis ke `HunterMode()` saat situasi mendeteksi kondisi duel satu lawan satu (`aliveEnemies == 1`). Pada fase krusial ini, bot secara *greedy* mengubah orientasi bertahannya menjadi serangan ofensif total jarak dekat guna memastikan musuh terakhir tereliminasi. Taktik agresif di akhir ini terbukti sangat efektif untuk mengunci raihan *Survival Bonus* penuh di setiap ronde pertandingan.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Empat Alternatif Solusi

4.1.1 Pseudocode Bot Utama

MULAI

DEKLARASI:

enemyX : real
enemyY : real
enemySpeed : real
enemyDirection : real
enemyEnergy : real = 100
enemyDetected : boolean = false
totalEnemies : integer = 30
deadEnemies : integer = 0
rnd : Random

PROGRAM UTAMA:

Jalankan bot

PROSEDUR Run

Atur warna body = (0x11, 0x11, 0x11)
Atur warna turret = (0xFF, 0x66, 0xCC)
Atur warna radar = (0x66, 0xCC, 0xFF)
Atur warna peluru = (0xFF, 0x99, 0xDD)
Atur warna gun = (0xCC, 0x33, 0x99)
Atur warna scan = (0x99, 0xDD, 0xFF)
Atur warna tracks = (0x33, 0x22, 0x33)

SELAMA bot berjalan LAKUKAN

Putar radar kanan 360 derajat

JIKA enemyDetected = false MAKA

Panggil prosedur MoveAround()

JIKA TIDAK

aliveEnemies = totalEnemies - deadEnemies

```

    JIKA aliveEnemies > 5 MAKA
        Panggil prosedur WaitingMode()
    ELSIF aliveEnemies > 1 MAKA
        Panggil prosedur PositioningMode()
    JIKA TIDAK
        Panggil prosedur HunterMode()
    ENDIF
ENDIF

ENDSELAMA
ENDPROSEDUR

PROSEDUR MoveAround
    Maju 80
    Belok kanan 30 derajat
    Maju 80
    Belok kiri 30 derajat
ENDPROSEDUR

PROSEDUR WaitingMode
    distance = jarak menuju (enemyX, enemyY)
    bearing = arah menuju (enemyX, enemyY)

    JIKA distance < 180 MAKA
        Belok kanan sebesar bearing + 180 derajat
        Maju 100
        Panggil prosedur ShootByDistance(distance)
    JIKA TIDAK
        Belok kanan 25 derajat
        Maju 70
        JIKA enemyEnergy < 35 ATAU distance < 300 MAKA
            Panggil prosedur ShootByDistance(distance)
        ENDIF
    ENDIF

    Putar radar kanan 60 derajat
ENDPROSEDUR

PROSEDUR PositioningMode
    distance = jarak menuju (enemyX, enemyY)

```

bearing = arah menuju (enemyX, enemyY)

JIKA distance > 300 MAKA

Belok kanan sebesar bearing + 45 derajat

Maju 90

ELSIF distance < 120 MAKA

Mundur 80

Belok kanan 40 derajat

JIKA TIDAK

Belok kanan sebesar bearing + 90 derajat

Maju 70

ENDIF

Panggil prosedur ShootByDistance(distance)

Putar radar kanan 45 derajat

ENDPROSEDUR

PROSEDUR HunterMode

distance = jarak menuju (enemyX, enemyY)

bearing = arah menuju (enemyX, enemyY)

Belok kiri sebesar bearing derajat

Maju 100

Panggil prosedur ShootByDistance(distance)

Putar radar kanan 45 derajat

ENDPROSEDUR

PROSEDUR ShootByDistance(distance)

JIKA enemyEnergy < 20 MAKA

firePower = 2

ELSIF distance < 120 MAKA

firePower = 3

ELSIF distance < 300 MAKA

firePower = 2

JIKA TIDAK

firePower = 1

ENDIF

Panggil prosedur PredictiveShoot(firePower)

ENDPROSEDUR

PROSEDUR PredictiveShoot(firePower)

 JIKA energi bot $\leq$ firePower MAKA

 STOP

 ENDIF

 distance = jarak menuju (enemyX, enemyY)

 bulletSpeed = $20 - (3 \times \text{firePower})$

 time = distance $\div$ bulletSpeed

 futureX = enemyX + $\sin(\text{enemyDirection} \times \pi \div 180) \times \text{enemySpeed} \times \text{time}$

 futureY = enemyY + $\cos(\text{enemyDirection} \times \pi \div 180) \times \text{enemySpeed} \times \text{time}$

 gunBearing = arah gun menuju (futureX, futureY)

 Putar gun kanan sebesar gunBearing

 JIKA nilai mutlak(gunBearing) < 15 derajat MAKA

 Tembak dengan kekuatan firePower

 ENDIF

ENDPROSEDUR

EVENT SaatMusuhTerdeteksi (OnScannedBot)

 enemyDetected = true

 enemyX = posisi X musuh

 enemyY = posisi Y musuh

 enemySpeed = kecepatan musuh

 enemyDirection = arah gerak musuh

 enemyEnergy = energi musuh

 radarBearing = arah radar menuju (enemyX, enemyY)

 Putar radar kanan sebesar radarBearing $\times 2$

ENDEVENT

EVENT SaatMenabrakDinding (OnHitWall)

 Mundur 70

 Belok kanan 90 derajat

ENDEVENT

EVENT SaatMenabrakBot (OnHitBot)

 JIKA energi bot > 30 MAKA

 Tembak dengan kekuatan 2

```

    ENDIF
    Mundur 80
    Belok kanan 70 derajat
ENDEVENT

EVENT SaatTerkenaPeluru (OnHitByBullet)
    sudutAcak = angka acak antara 45 sampai 99
    Belok kanan sebesar sudutAcak derajat
    Maju 100
ENDEVENT

EVENT SaatBotMusuhMati (OnBotDeath)
    deadEnemies = deadEnemies + 1
    enemyDetected = false
ENDEVENT

SELESAI

```

4.1.2 Pseudocode Bot Alternatif 1

MULAI

DEKLARASI:

```

    enemyX      : real
    enemyY      : real
    enemyDistance : real
    enemyEnergy  : real = 100
    enemyDetected : boolean = false
    rnd         : Random

```

PROGRAM UTAMA:

Jalankan bot

PROSEDUR Run

```

    Atur warna body   = (0x33, 0x33, 0xCC)
    Atur warna turret = (0x99, 0x33, 0xFF)
    Atur warna radar  = (0xFF, 0xFF, 0x66)
    Atur warna peluru = (0xFF, 0xCC, 0x00)
    Atur warna gun     = (0x66, 0x00, 0xCC)
    Atur warna scan    = (0xCC, 0xCC, 0xFF)
    Atur warna tracks  = (0x22, 0x22, 0x44)

```

Panggil prosedur MoveToCorner()

SELAMA bot berjalan LAKUKAN

JIKA enemyDetected = true MAKA

Panggil prosedur AttackMode()

JIKA TIDAK

Panggil prosedur SurvivalMode()

ENDIF

Putar radar kanan 360 derajat

ENDSELAMA

ENDPROSEDUR

PROSEDUR MoveToCorner

targetX = 60

targetY = 60

bearing = arah menuju (targetX, targetY)

distance = jarak menuju (targetX, targetY)

Belok kanan sebesar bearing

Maju sejauh distance

ENDPROSEDUR

PROSEDUR SurvivalMode

Maju 80

Belok kanan 30 derajat

Maju 80

Belok kiri 30 derajat

ENDPROSEDUR

PROSEDUR AttackMode

radarBearing = arah radar menuju (enemyX, enemyY)

Putar radar kanan sebesar radarBearing $\times$ 2

gunBearing = arah gun menuju (enemyX, enemyY)

Putar gun kanan sebesar gunBearing

firePower = Panggil fungsi CalculateFirePower()

```
JIKA enemyDistance < 250 ATAU enemyEnergy < 30 MAKA
    Tembak dengan kekuatan firePower
ENDIF
```

```
Maju 60
Belok kanan 25 derajat
ENDPROSEDUR
```

```
FUNGSI CalculateFirePower
    JIKA enemyEnergy < 20 MAKA
        Kembalikan 3
    ELSIF enemyDistance < 100 MAKA
        Kembalikan 3
    ELSIF enemyDistance < 250 MAKA
        Kembalikan 2
    JIKA TIDAK
        Kembalikan 1
    ENDIF
ENDFUNGSI
```

```
EVENT SaatMusuhTerdeteksi (OnScannedBot)
    enemyDetected = true
    enemyX        = posisi X musuh
    enemyY        = posisi Y musuh
    enemyDistance = jarak menuju (enemyX, enemyY)
    enemyEnergy   = energi musuh
ENDEVENT
```

```
EVENT SaatMenabrakDinding (OnHitWall)
    Mundur 60
    Belok kanan 90 derajat
ENDEVENT
```

```
EVENT SaatTerkenaPeluru (OnHitByBullet)
    sudutAcak = angka acak antara 45 sampai 99
    Belok kanan sebesar sudutAcak derajat
    Maju 100
ENDEVENT
```

```
EVENT SaatMenabrakBot (OnHitBot)
    Tembak dengan kekuatan 3
```



```

Mundur 50
Belok kanan 90 derajat
ENDEVENT

EVENT SaatBotMusuhMati (OnBotDeath)
    enemyDetected = false
ENDEVENT

SELESAI

```

4.1.3 Pseudocode Bot Alternatif 2

```

MULAI

DEKLARASI:
    enemyX      : real
    enemyY      : real
    enemyDetected : boolean = false
    rnd         : Random

PROGRAM UTAMA:
    Jalankan bot

PROSEDUR Run
    Atur warna body   = (0x11, 0x11, 0x11)
    Atur warna turret = (0xFF, 0x66, 0x00)
    Atur warna radar  = (0xFF, 0x22, 0x22)
    Atur warna peluru = (0xFF, 0x99, 0x00)
    Atur warna gun    = (0xCC, 0x33, 0x00)
    Atur warna scan   = (0xFF, 0x44, 0x44)
    Atur warna tracks = (0x33, 0x33, 0x33)

    SELAMA bot berjalan LAKUKAN

        Putar radar kanan 360 derajat

        JIKA energi bot > 70 MAKA
            Panggil prosedur MoveToEdge()
        ELSIF energi bot > 30 MAKA
            Panggil prosedur ZigZag()
        JIKA TIDAK

```

```
Panggil prosedur ChaosMove()  
ENDIF
```

```
ENDSELAMA  
ENDPROSEDUR
```

```
PROSEDUR MoveToEdge  
  Belok kanan 30 derajat  
  Maju 50  
ENDPROSEDUR
```

```
PROSEDUR ZigZag  
  Maju 80  
  Belok kanan 45 derajat  
  Maju 80  
  Belok kiri 45 derajat  
ENDPROSEDUR
```

```
PROSEDUR ChaosMove  
  arahGerak = acak antara 1 atau -1  
  sudutBelok = angka acak antara 30 sampai 89  
  jarakMaju = angka acak antara 50 sampai 149  
  
  Belok kanan sebesar sudutBelok  $\times$  arahGerak derajat  
  Maju sejauh jarakMaju  $\times$  arahGerak  
ENDPROSEDUR
```

```
EVENT SaatMusuhTerdeteksi (OnScannedBot)  
  enemyX      = posisi X musuh  
  enemyY      = posisi Y musuh  
  enemyDetected = true  
  
  distance = jarak menuju (enemyX, enemyY)  
  gunBearing = arah gun menuju (enemyX, enemyY)  
  Putar gun kanan sebesar gunBearing
```

```
JIKA energi bot > 70 MAKA
```

```
  JIKA distance < 150 MAKA  
    Tembak dengan kekuatan 3
```

```

    Maju 50
    JIKA TIDAK
        Tembak dengan kekuatan 1
    ENDIF

    ELSIF energi bot > 30 MAKA

        JIKA distance < 200 MAKA
            Tembak dengan kekuatan 2
        JIKA TIDAK
            Tembak dengan kekuatan 1
        ENDIF

    JIKA TIDAK
        Tembak dengan kekuatan 1
        Mundur 100
        Belok kanan 90 derajat
    ENDIF

ENDEVENT

EVENT SaatMenabrakDinding (OnHitWall)
    Mundur 50
    Belok kanan 90 derajat
ENDEVENT

EVENT SaatMenabrakBot (OnHitBot)
    JIKA energi bot > 50 MAKA
        Tembak dengan kekuatan 3
    JIKA TIDAK
        Mundur 100
        Belok kanan 90 derajat
    ENDIF
ENDEVENT

SELESAI

```

4.1.4 Pseudocode Bot Alternatif 3

```

MULAI

```

DEKLARASI:

enemyX : real
enemyY : real
enemySpeed : real
enemyDirection: real
enemyDetected : boolean = false
flankSide : integer = 1
rnd : Random

PROGRAM UTAMA:

Jalankan bot

PROSEDUR Run

Atur warna body = (0x66, 0x33, 0xCC)
Atur warna turret = (0x99, 0xCC, 0xFF)
Atur warna radar = (0xFF, 0x66, 0xCC)
Atur warna peluru = (0xFF, 0x99, 0xDD)
Atur warna gun = (0xCC, 0x99, 0xFF)
Atur warna scan = (0xFF, 0xCC, 0xEE)
Atur warna tracks = (0x33, 0x22, 0x55)

SELAMA bot berjalan LAKUKAN

JIKA enemyDetected = false MAKA

Putar radar kanan 360 derajat

JIKA TIDAK

JIKA energi bot > 30 MAKA

Panggil prosedur FlankMode()

JIKA TIDAK

Panggil prosedur EscapeMode()

ENDIF

ENDIF

ENDSELAMA

ENDPROSEDUR

PROSEDUR FlankMode

distance = jarak menuju (enemyX, enemyY)

bearing = arah menuju (enemyX, enemyY)

```

    JIKA distance > 250 MAKA
        Panggil prosedur ApproachFromSide(bearing)
    ELSIF distance < 100 MAKA
        Belok kanan sebesar flankSide × 90 derajat
        Mundur 80
    JIKA TIDAK
        Panggil prosedur OrbitEnemy(bearing)
    ENDIF
ENDPROSEDUR

```

```

PROSEDUR ApproachFromSide(bearing)
    Belok kanan sebesar bearing + (flankSide × 90) derajat
    Maju 100
    Putar radar kanan 45 derajat
ENDPROSEDUR

```

```

PROSEDUR OrbitEnemy(bearing)
    Belok kanan sebesar bearing + (flankSide × 90) derajat
    Maju 50
    Putar radar kanan 45 derajat
ENDPROSEDUR

```

```

PROSEDUR EscapeMode
    bearing = arah menuju (enemyX, enemyY)
    Belok kanan sebesar bearing + 180 derajat
    Maju 150
    Putar radar kanan 90 derajat
ENDPROSEDUR

```

```

PROSEDUR PredictiveShoot(firePower)
    bulletSpeed = 20 - (3 × firePower)
    distance = jarak menuju (enemyX, enemyY)
    time = distance ÷ bulletSpeed

    futureX = enemyX + sinus(enemyDirection ×  $\pi$  ÷ 180) × enemySpeed × time
    futureY = enemyY + cosinus(enemyDirection ×  $\pi$  ÷ 180) × enemySpeed × time

    gunBearing = arah gun menuju (futureX, futureY)
    Putar gun kanan sebesar gunBearing

```

Tembak dengan kekuatan firePower
ENDPROSEDUR

EVENT SaatMusuhTerdeteksi (OnScannedBot)

enemyX = posisi X musuh
enemyY = posisi Y musuh
enemySpeed = kecepatan musuh
enemyDirection = arah gerak musuh
enemyDetected = true

radarBearing = arah radar menuju (enemyX, enemyY)
Putar radar kanan sebesar radarBearing $\times$ 2

distance = jarak menuju (enemyX, enemyY)

JIKA energi bot > 50 MAKA

JIKA distance < 150 MAKA
Panggil prosedur PredictiveShoot(3)
JIKA TIDAK
Panggil prosedur PredictiveShoot(2)
ENDIF

ELSIF energi bot > 30 MAKA
Panggil prosedur PredictiveShoot(1)

JIKA TIDAK
Panggil prosedur PredictiveShoot(1)
ENDIF

JIKA angka acak dari 0 sampai 9 = 0 MAKA
flankSide = flankSide $\times$ -1
ENDIF
ENDEVENT

EVENT SaatMenabrakDinding (OnHitWall)

Mundur 50
flankSide = flankSide $\times$ -1
sudutAcak = angka acak antara 45 sampai 89
Belok kanan sebesar sudutAcak $\times$ flankSide derajat

ENDEVENT

EVENT SaatMenabrakBot (OnHitBot)

JIKA energi bot > 30 MAKA

Belok kanan sebesar flankSide $\times$ 90 derajat

Mundur 80

Panggil prosedur PredictiveShoot(2)

JIKA TIDAK

Mundur 100

sudutAcak = angka acak antara 90 sampai 179

Belok kanan sebesar sudutAcak derajat

ENDIF

ENDEVENT

EVENT SaatBotMusuhMati (OnBotDeath)

enemyDetected = false

flankSide = flankSide $\times$ -1

ENDEVENT

EVENT SaatTerkenaPeluru (OnHitByBullet)

flankSide = flankSide $\times$ -1

Belok kanan sebesar flankSide $\times$ 45 derajat

Maju 60

ENDEVENT

SELESAI

4.2 Struktur Data, Fungsi, dan Prosedur

4.2.1 Struktur Data

Secara spesifik, struktur data yang diterapkan di dalam *source code* program kelompok kami diimplementasikan untuk mendukung fungsi seleksi algoritma *Greedy* pada masing-masing bot, yang terbagi menjadi beberapa tipe berkas data berikut:

- **Variabel bertipe double (Bilangan Pecahan Presisi):** Tipe data ini digunakan secara intensif di seluruh variasi bot untuk menyimpan data numerik spasial dengan tingkat akurasi tinggi. Pada **Alt 1 (Survival Bot)**, tipe ini menyimpan batas koordinat sudut aman. Pada **Alt 2 (Adaptive Bot)**, digunakan untuk melacak sisa kalkulasi persentase energi. Sementara pada **Alt 3 (Flanker Bot)** dan **Main Bot (Opportunist Bot)**, variabel

double bertindak sebagai penyimpan data krusial seperti koordinat posisi musuh (enemyX dan enemyY), jarak linear (enemyDistance), serta kecepatan dan arah sudut radar (enemySpeed dan enemyDirection) guna melakukan kalkulasi penembakan prediktif berbasis fisika peluru.

- **Objek bertipe Random (Generator Nilai Acak):** Struktur data objek ini digunakan khusus melalui instansiasi `rnd = new Random()`. Objek ini dimanfaatkan secara eksklusif oleh **Alt 2 (Adaptive Bot)** di dalam perulangan taktisnya. Fungsi utamanya adalah memproduksi nilai acak secara instan untuk memicu keputusan arah manuver tidak terduga (ChaosMove) ketika kondisi fisik atau energi bot sedang kritis, sehingga pola pelarian bot menjadi sangat acak dan sulit dibaca oleh radar lawan.
- **Variabel bertipe int (Bilangan Bulat):** Tipe data ini digunakan untuk menyimpan nilai status atau indeks bilangan bulat. Pada **Alt 3 (Flanker Bot)**, variabel int diterapkan pada parameter `flankSide` yang bernilai spesifik 1 atau -1 sebagai penanda arah rotasi kepungan (searah atau berlawanan jarum jam). Sedangkan pada **Main Bot (Opportunist Bot)**, tipe data ini memegang peranan paling vital sebagai basis penentu keputusan *greedy* global melalui variabel statistik jumlah musuh, seperti `totalEnemies`, `deadEnemies`, dan sisa musuh hidup (`aliveEnemies`).
- **Variabel bertipe boolean (Logika True/False):** Struktur data logika ini digunakan di seluruh variasi bot, khususnya melalui variabel `enemyDetected` sebagai penanda (*flag*) kondisi radar aktif. Struktur data ini berfungsi mendeteksi secara instan apakah radar sedang berhasil mengunci target atau kehilangan jejak, sehingga sistem bot dapat mengambil keputusan *greedy* lokal kapan harus mengeksekusi mode menjelajah/bertahan (SurvivalMode/MoveAround) atau langsung masuk ke mode menyerang (AttackMode).

4.2.2 Fungsi

Fungsi-fungsi di dalam *source code* program kami bertugas untuk melakukan kalkulasi geometris, trigonometri, dan perhitungan matematis secara instan terhadap status lingkungan pertempuran. Seluruh fungsi ini wajib mengembalikan suatu nilai spesifik (*return value*) yang akan digunakan oleh sistem utama untuk mengambil keputusan taktis secara *greedy*.

Secara spesifik, fungsi-fungsi yang diterapkan di dalam program kami meliputi:

- **CalculateFirePower() (Eksklusif pada Alt 1 - Survival Bot):**
Fungsi ini mengevaluasi kondisi jarak linear (*enemyDistance*) dan sisa energi musuh (*enemyEnergy*). Fungsi secara *greedy* akan mengembalikan nilai daya tembak maksimal 3.0 jika musuh dalam posisi sangat dekat (≤ 100 unit) atau sekarat (≤ 20) demi mengamankan poin eliminasi instan. Jika musuh menjauh atau dalam kondisi stabil, fungsi secara efisien akan menurunkan nilai kembalian daya menjadi 2.0 atau 1.0 guna menghemat kapasitas energi internal bot.
- **DistanceTo(double x, double y) (Digunakan oleh Seluruh Bot):** Fungsi geometri yang menerapkan rumus jarak *Euclidean* untuk menghitung jarak linear dari posisi koordinat tank. Nilai kembalian dari fungsi ini sangat krusial bagi **Alt 2 (Adaptive Bot)**, **Alt 3 (Flanker Bot)**, dan **Main Bot (Opportunist Bot)** sebagai fungsi seleksi pembatas transisi jarak aman aksi pertempuran.
- **BearingTo(), RadarBearingTo(), dan GunBearingTo() (Digunakan oleh Alt 3 dan Main Bot):** Kumpulan fungsi trigonometri lanjut yang bertugas mengalkulasi dan mengembalikan nilai sudut relatif (dalam satuan derajat antara -180° hingga 180°) antara arah hadap tubuh, radar, maupun moncong meriam bot kita terhadap koordinat musuh saat ini. Nilai sudut kembalian ini memegang peranan paling vital bagi fungsi penembakan prediktif pada **Alt 3 (Flanker Bot)** dan fungsi penguncian target pada **Main Bot (Opportunist Bot)** agar rotasi perputaran meriam dapat mengeksekusi tembakan secara presisi tanpa meleset.

4.2.3 Prosedur

Prosedur di dalam kode program diwujudkan dalam bentuk *method* bertipe void yang bertugas untuk mengeksekusi serangkaian instruksi gerakan mekanis atau aksi penembakan secara langsung tanpa mengembalikan nilai data (*no return value*). Berdasarkan analisis *source code*, prosedur yang diterapkan pada program kelompok kami dapat dikelompokkan menjadi empat kategori utama berikut:

1. Prosedur Manajemen Mode Pertempuran (*Battle Mode Management*)

Prosedur ini bertanggung jawab penuh sebagai pengatur transisi kondisi taktis atau status kesiapan tempur bot di arena:

- **Alt 1 (Survival Bot):** Menggunakan prosedur `AttackMode()` untuk mengunci pergerakan radar pada satu target dan `SurvivalMode()` untuk mengeksekusi pergerakan reguler di area aman.
- **Alt 3 (Flanker Bot):** Menggunakan prosedur `FlankMode()` untuk menginisialisasi taktik mengepung musuh dari sisi samping dan `EscapeMode()` untuk melarikan diri membelakangi posisi lawan saat terdesak.
- **Main Bot (Opportunist Bot):** Mengatur transisi kondisi pertempuran secara makro melalui tiga prosedur dinamis, yaitu `WaitingMode()` untuk menjauh secara defensif di awal laga, `PositioningMode()` untuk menjaga jarak ideal di fase transisi, dan `HunterMode()` untuk melakukan serbuan ofensif agresif saat duel satu lawan satu.

2. Prosedur Manuver dan Pergerakan Tubuh (*Movement & Maneuver*)

Prosedur ini mengatur pergerakan fisik mekanis dari roda atau *chassis* tangki robot di atas koordinat arena:

- **Alt 1 (Survival Bot):** Menerapkan prosedur `MoveToCorner()` untuk memaksa bot bergerak lurus menuju koordinat sudut aman \$(60, 60)\$ di awal ronde.
- **Alt 2 (Adaptive Bot):** Memiliki variasi prosedur pergerakan paling dinamis, meliputi `MoveToEdge()` (bergerak ke tepian perbatasan), `ZigZag()` (gerakan mengecoh berbelok-belok), dan `ChaosMove()` yang memicu manuver acak via perintah `rnd.Next()` saat energi kritis.
- **Alt 3 (Flanker Bot):** Menggunakan prosedur `OrbitEnemy()` dan `ApproachFromSide()` untuk mengitari posisi koordinat target secara tegak lurus.

3. Prosedur Ofensif dan Penembakan Khusus (*Offensive & Custom Shooting*)

Prosedur ini mengatur kalkulasi sistem persenjataan dan pelepasan peluru energi dari moncong meriam bot:

- **Alt 3 (Flanker Bot) & Main Bot (Opportunist Bot):** Menerapkan prosedur `PredictiveShoot(double firePower)`, yaitu prosedur penembakan tingkat lanjut yang mengukur estimasi waktu tempuh peluru untuk memproyeksikan koordinat masa depan musuh (`futureX` dan `futureY`), menyelaraskan posisi meriam secara otomatis, lalu melepaskan tembakan presisi.

- **Main Bot (Opportunist Bot):** Menggunakan prosedur tambahan ShootByDistance(double distance) sebagai prosedur perantara untuk menetapkan besaran skala daya tembak (*fire power*) yang paling efisien sebelum perintah menembak dieksekusi.

4. Prosedur Interupsi Acara (*Event Handler API*)

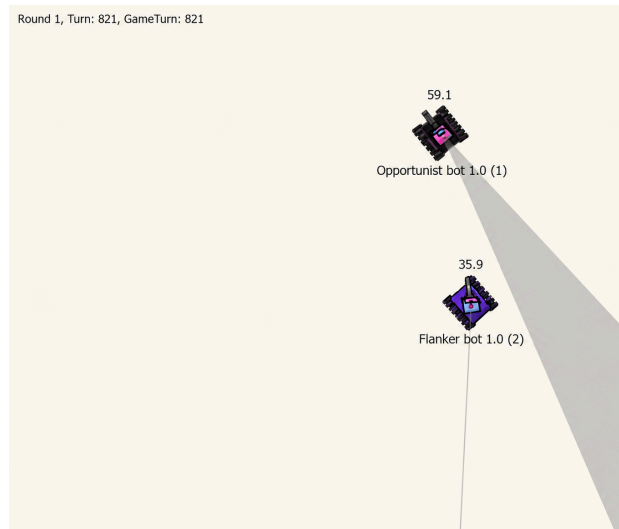
Merupakan kumpulan prosedur otomatis bawaan API Robocode Tank Royale yang bekerja berbasis interupsi (*event-driven*). Prosedur ini bertugas memperbarui seluruh variabel struktur data global secara *real-time* saat terjadi suatu peristiwa di arena, yang diimplementasikan secara seragam oleh seluruh bot kami:

- OnScannedBot(): Dieksekusi otomatis saat radar mendeteksi objek musuh.
- OnHitWall(): Dieksekusi otomatis sebagai algoritma penyelamatan saat bodi tank menabrak dinding arena.
- OnHitBot(): Dieksekusi otomatis saat terjadi berbenturan fisik dengan robot lain.
- OnBotDeath(): Dieksekusi otomatis saat ada robot musuh yang hancur guna memperbarui statistik jumlah musuh.

4.3 Pengujian Program

Pengujian dilakukan sebanyak tiga kali dalam format pertandingan 1 vs 1 antara bot utama (*Opportunist*) dengan masing-masing bot alternatif. Setiap pertandingan terdiri dari 4 round. Hasil skor merupakan akumulasi dari seluruh round yang dimainkan.

4.3.1 Pengujian 1



Gambar 4.1 Hasil Pengujian 1

Tabel 4.1 Analisis Pengujian 1

Kategori	Opportunist bot	Flanker bot
Total Score	245	168
Survival Score	180	90
Survival Bonus	40	20
Bullet Damage	15	42
Bullet Bonus	10	8
Ram Damage	0	5
Ram Bonus	0	3
Juara 1 (round)	4	1
Juara 2 (round)	1	4

4.3.2 Pengujian 2

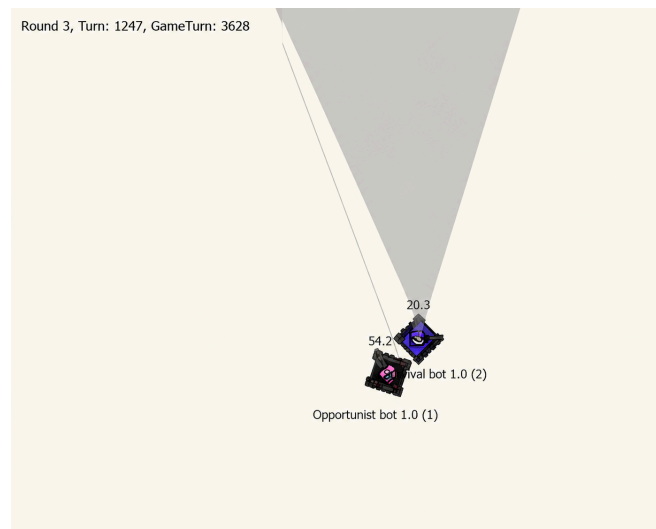


Gambar 4.2 Hasil Pengujian 2

Tabel 4.2 Analisis Pengujian 2

Kategori	Opportunist bot	Survival bot
Total Score	256	165
Survival Score	200	120
Survival Bonus	40	30
Bullet Damage	18	8
Bullet Bonus	6	2
Ram Damage	4	2
Ram Bonus	0	0
Juara 1 (round)	3	1
Juara 2 (round 2)	1	3

4.3.3 Pengujian 3

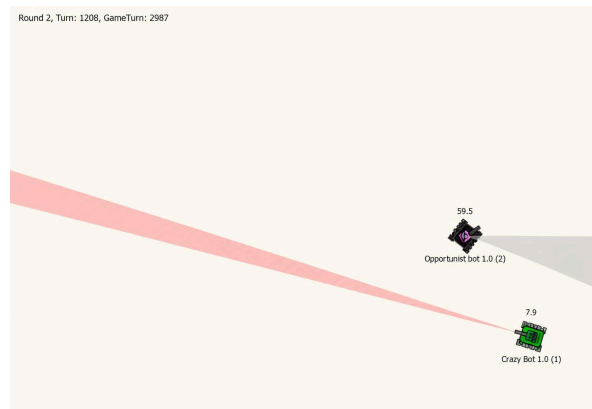


Gambar 4.3 Hasil Pengujian

Tabel 4.3 Analisis Pengujian 3

Kategori	Opportunist bot	Adaptive bot
Total Score	278	143
Survival Score	160	40
Survival Bonus	35	0
Bullet Damage	48	55
Bullet Bonus	10	4
Ram Damage	25	18
Ram Bonus	1	0
Juara 1 (round)	3	1
Juara 2 (round 2)	1	3

4.3.4 Pengujian 4



Gambar 4.4 Hasil Pengujian 4

Tabel 4.4 Analisis Pengujian 4

Kategori	Opportunist bot	Crazy bot
Total Score	150	95
Survival Score	100	0
Survival Bonus	20	88
Bullet Damage	30	0
Bullet Bonus	2	7
Ram Damage	1	0
Ram Bonus	0	1
Juara 1 (round)	2	1
Juara 2 (round 2)	1	2

4.4 Analisis dan Hasil Pengujian

Berdasarkan hasil keempat pengujian yang telah dilakukan, strategi greedy yang diterapkan oleh Opportunist Bot sebagai bot utama terbukti sangat berhasil dalam mengamankan

poin bertahan hidup (survival) dan mendominasi kemenangan secara konsisten pada sebagian besar skenario arena. Pada Pengujian 1 melawan Flanker Bot, Opportunist Bot sukses menyapu bersih Survival Score sebesar 180 poin dan Survival Bonus 40 poin dengan meraih 4 gelar Juara 1 dari total 5 ronde, unggul mutlak dengan total skor 245 berbanding 168 poin. Keunggulan ini dicapai meskipun Bullet Damage Opportunist (15 poin) jauh di bawah Flanker (42 poin) dan Ram Damage Opportunist 0 poin, membuktikan bahwa strategi greedy sangat efektif dalam menghindari pertempuran langsung dan memaksimalkan poin bertahan hidup tanpa perlu agresivitas tinggi.

Pada Pengujian 2 melawan Survival Bot, Opportunist Bot kembali mendominasi dengan Survival Score maksimal 200 poin dan Survival Bonus 40 poin, serta total skor akhir 256 berbanding 165 poin. Menariknya, pada pengujian ini Opportunist mulai menunjukkan peningkatan agresivitas dengan Bullet Damage 18 poin (naik dari 15) dan Ram Damage 4 poin (sebelumnya 0), mengindikasikan bahwa Opportunist Bot mampu menyesuaikan tingkat agresivitas saat menghadapi lawan yang juga bertahan, namun tetap mempertahankan fondasi strategi greedy-nya.

Memasuki Pengujian 3 melawan Adaptive Bot, Opportunist Bot menghadapi tantangan terbesar. Meskipun masih unggul total skor 278 berbanding 143 poin, Survival Score Opportunist turun drastis menjadi 160 poin dari sebelumnya 200 poin. Penyebabnya adalah Bullet Damage Adaptive Bot (55 poin) yang melampaui Opportunist (48 poin), memaksa Opportunist keluar dari zona nyamannya untuk bertarung. Namun sebagai respons, Opportunist meningkatkan Ram Damage hingga 25 poin dan meraih Ram Bonus 1 poin — sebuah bukti bahwa Opportunist Bot mampu beradaptasi ketika tekanan meningkat, meskipun dengan mengorbankan poin bertahan hidup.

Kondisi tersebut dipertegas pada Pengujian 4 saat melawan Crazy bot, di mana dominasi Opportunist Bot kembali tertekan oleh pergerakan lawan yang tidak terprediksi. Meskipun Opportunist Bot masih memenangkan total poin tertinggi dengan total skor 150 berbanding 95 poin, *Survival Score*-nya tertahan di angka 100 poin dan ia harus merelakan 2 *round* jatuh ke tangan Crazy bot sebagai Juara 2. Menariknya, pada fase ini Opportunist Bot dipaksa keluar dari zona pasifnya untuk melakukan perlawanan defensif, yang ditunjukkan dengan munculnya angka *Bullet Damage* sebesar 30 poin dan sedikit *Ram Damage* sebesar 1 poin.

Secara keseluruhan, sebagai bot utama, Opportunist Bot dengan strategi greedy menunjukkan performa yang dominan dengan kelebihan utama berupa efektivitas tinggi melawan bot dengan pola tetap seperti Flanker dan Survival, kemampuan meraih Survival Score maksimal 200 poin, serta konsistensi memenangkan total skor di semua pengujian. Namun demikian, kelemahan strategi ini mulai terlihat ketika dihadapkan pada lawan yang dinamis dan tidak terprediksi. Survival Score menurun drastis saat melawan Adaptive Bot dan Crazy Bot, sementara sifat pasif yang cenderung ditunjukkan Opportunist terbukti menjadi celah yang dapat dieksploitasi. Pola greedy juga rentan terhadap taktik survival murni seperti yang dilakukan Crazy Bot yang mampu mengganggu ritme permainan.

Yang menarik untuk dicatat, peningkatan Bullet Damage Opportunist dari 15 poin pada Pengujian 1 menjadi 30 poin pada Pengujian 4, serta peningkatan Ram Damage dari 0 menjadi 25 poin pada Pengujian 3, menunjukkan bahwa Opportunist Bot sesungguhnya memiliki kapasitas untuk belajar dan beradaptasi. Hanya saja, mekanisme adaptasi ini perlu diotomatisasi lebih awal agar tidak kehilangan poin bertahan hidup di ronde-ronde kritis. Dengan demikian, kesimpulan akhir dari seluruh rangkaian pengujian ini adalah bahwa strategi greedy pada Opportunist Bot terbukti dominan secara keseluruhan dan sangat efektif dalam mengamankan kemenangan, namun memiliki celah signifikan terhadap lawan dengan pola dinamis dan tidak konvensional. Untuk pengembangan ke depan, strategi greedy perlu dimodifikasi menjadi model hybrid yang tetap mempertahankan fondasi bertahan tetapi dilengkapi dengan respons agresif yang terukur serta mekanisme deteksi pola lawan, sehingga Opportunist Bot dapat mempertahankan dominasinya di semua skenario arena.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian keempat model bot kelompok TinkTank pada platform Robocode Tank Royale, dapat diambil beberapa kesimpulan utama mengenai penerapan algoritma *Greedy* sebagai berikut:

1. **Efektivitas *Greedy* dalam Aspek Pertahanan (*Survival Rate*):**

Pendekatan *greedy* yang berfokus pada keuntungan lokal jangka pendek terbukti sangat sukses dalam mengamankan poin bertahan hidup. Dengan memprioritaskan keselamatan diri di awal pertandingan—seperti mengunci posisi di pojok lapangan pada **Alt 1 (Survival Bot)** atau menjaga jarak aman global pada **Main Bot (Opportunist Bot)**—bot mampu menghindari eliminasi dini dalam situasi arena yang ramai dan mengumpulkan *Survival Score* serta *Survival Bonus* secara maksimal.

2. **Kelemahan *Greedy* pada Efisiensi dan Efektivitas Ofensif:**

Sifat dasar algoritma *greedy* yang cenderung mencari aman memunculkan kelemahan pada aspek ofensif, di mana bot yang terlalu fokus bertahan akan menghasilkan nilai kerusakan senjata (*Bullet Damage*) yang sangat minim pada fase-fase awal ronde. Sebaliknya, jika perilaku *greedy* dialihkan sepenuhnya untuk mengejar daya rusak secara agresif berbasis kondisi internal seperti pada **Alt 2 (Adaptive Bot)** atau pengepungan ofensif pada **Alt 3 (Flanker Bot)**, bot menjadi sangat boros energi akibat penembakan konstan dan rentan tereliminasi lebih cepat di awal laga.

3. **Keunggulan Strategi Dinamis Berbasis Informasi Global:**

Melalui seluruh hasil analisis dan rangkaian pengujian, **Main Bot (Opportunist Bot)** ditetapkan sebagai strategi terbaik karena berhasil mengatasi kekakuan sifat *greedy*. Dengan memanfaatkan transisi informasi global (jumlah sisa musuh yang hidup) untuk membagi mode pertempuran secara dinamis (*WaitingMode*, *PositioningMode*, dan *HunterMode*), *Main Bot* mampu menyelaraskan aspek efisiensi energi di awal laga yang ramai dan efektivitas serangan agresif pada saat duel satu lawan satu. Pendekatan hibrida

ini terbukti menjadi formula paling konsisten untuk menghasilkan total skor akumulasi tertinggi.

4. **Ketergantungan Performa Terhadap Karakteristik Pergerakan Lawan:**

Strategi *greedy* memiliki performa yang sangat bergantung pada tipe musuh yang dihadapi di arena. Strategi bertipe statis dan ofensif linear akan bekerja dengan sangat optimal dan dominan saat menghadapi lawan dengan pola gerakan yang kaku dan terprediksi seperti pada **Alt 1 (Survival Bot)**. Namun, performa tersebut akan mengalami tantangan dan penurunan efisiensi ketika dihadapkan dengan tipe musuh yang bergerak dinamis, memiliki manuver acak, atau membaca masa depan seperti pada **Alt 2 (Adaptive Bot)** dan **Alt 3 (Flanker Bot)**, karena sifat keputusan *greedy* lokal yang cenderung konstan kurang mampu memprediksi perubahan taktik lawan yang berubah-ubah secara instan.

5.2 Saran

Berdasarkan hasil analisis, identifikasi kelemahan, serta kesimpulan dari rangkaian pengujian yang telah dilakukan terhadap seluruh model bot kelompok TinkTank, berikut adalah beberapa saran konstruktif yang dapat diterapkan untuk pengembangan sistem kecerdasan otonom bot di masa mendatang:

1. **Penerapan Konsep *Hybrid Greedy* dan Algoritma Heuristik Lanjut:**

Untuk mengatasi sifat *greedy* lokal yang cenderung kaku dan pasif pada **Main Bot (Opportunist Bot)** saat menghadapi situasi arena yang dinamis, sistem kecerdasan bot dapat dikembangkan menjadi sistem hibrida (*hybrid system*) dengan mengintegrasikan *Fuzzy Logic* atau Algoritma Genetika (*Genetic Algorithm*). Kombinasi ini akan membantu bot dalam menentukan bobot nilai fungsi seleksi (*heuristic weight*) secara lebih fleksibel untuk menentukan kapan harus bertransisi ke mode bertahan atau menyerang berdasarkan perubahan kondisi *real-time* di arena.

2. **Optimasi Konsumsi Energi pada Integrasi Tembakan Prediktif:**

Metode tembakan prediktif (*predictive shoot*) milik **Alt 3 (Flanker Bot)** yang terbukti sangat efektif menghasilkan poin kerusakan masif disarankan untuk diintegrasikan ke dalam sistem persenjataan **Main Bot (Opportunist Bot)**, namun dengan pembatasan

aturan *greedy* energi yang ketat. Bot masa depan disarankan hanya mengaktifkan perhitungan matematika prediksi lintasan berdaya penuh (Fire(3.0)) jika tingkat akurasi radar berada di atas persentase tertentu dan sisa energi internal bot sendiri masih berada di atas batas aman (misalnya $>50\%$).

3. **Pengembangan Sistem Sensor dan Pelacakan Multi-Target (*Advanced Radar*):**

Saat ini, keputusan transisi mode pertempuran pada **Main Bot (Opportunist Bot)** baru didasarkan pada jumlah total musuh yang hidup secara global (*aliveEnemies*). Sistem radar sebaiknya ditingkatkan agar mampu melacak dan menyimpan data koordinat, kecepatan, serta sisa energi dari setiap objek musuh secara spesifik ke dalam struktur data *array* atau *list*. Dengan demikian, bot dapat melakukan *micro-management* untuk memilih target lokal yang paling lemah atau paling dekat secara instan (*target lock-on prioritization*).

4. **Implementasi Strategi *Evasion* (Menghindar) yang Lebih Cerdas:**

Mekanisme pertahanan berbasis manajemen jarak linier pada **Main Bot (Opportunist Bot)** perlu dikombinasikan dengan pergerakan adaptif seperti pola lingkaran (*orbiting*) atau *anti-gravity movement* agar arah gerak bodi tank tidak mudah terbaca. Peningkatan ini sangat penting agar bot tidak sekadar mundur secara lurus, yang membuatnya rentan terkena tembakan telak dari bot-bot lawan yang bertipe pelacak lintasan masa depan seperti pada karakteristik **Alt 3 (Flanker Bot)**.

LAMPIRAN

Link GitHub: [Link Repository Github Tubes TinkTank](#)

Link Video Demonstrasi: [Link Video Demontrasi Tubes TinkTank](#)

DAFTAR PUSTAKA

- E-Jurnal Mikroskil. (n.d.). “Komponen-Komponen Algoritma Greedy.” <https://ejurnal.mikroskil.ac.id/>
- E-Jurnal UNAMA. (n.d.). “Implementasi Strategi Greedy pada Bot Robocode.” <https://ejournal.unama.ac.id/>
- Jurnal Ilmubersama. (n.d.). “Analisis Algoritma Greedy dalam Pengambilan Keputusan Real-Time.” <https://jurnal.ilmubersama.com/index.php/sudo/article/view/8/16>
- Jurnal Kaputama. (n.d.). “Penerapan Algoritma Greedy pada Permainan Robocode.” <https://jurnal.kaputama.ac.id/>
- Jurnal PNJ. (n.d.). “Fungsi Seleksi dalam Algoritma Greedy.” <https://jurnal.pnj.ac.id/>
- Jurnal UMNU. (n.d.). “Karakteristik dan Penerapan Algoritma Greedy.” <https://jurnal.umnu.ac.id/>
- Munir, Rinaldi. (2016). *Algoritma Greedy*. Bandung: Informatika Bandung.
- Robocode Dev. (2024). “Robocode Tank Royale Documentation - Bot API for .NET.” <https://book.robocode.dev/>
- Robocode Dev. (2024). “Robocode Tank Royale - Starter Guide.” <https://robocode.dev/>
- Robocode SourceForge. (2024). “Robocode - Robot Battle Code”. <https://robocode.sourceforge.io/>
- Robocode Tank Royale GitHub. (2024). “Tank Royale Engine Repository.” <https://github.com/robocode-dev/tank-royale>
- RoboWiki. (2024). “Robocode Strategies and Techniques.” <https://robowiki.net/>
- Sianaeng, Eviyanti Rudni; Kewo, Angreine; & Suwanto, Thomas Christian. (2015). “Aplikasi Penentuan Rute Perjalanan di Kota Manado Berbasis Web dengan Menggunakan Algoritma Greedy (Studi Kasus Teknik Informatika Unika De La Salle Manado).” Skripsi, Universitas Katolik De La Salle Manado. <https://repo.unikadelasalle.ac.id/3439/>